Real-Time Medicine Inventory, Batch and Expiry Tracking System

1. Registration — `register.html`

FORM ONLY

REGISTER

Full Name [_____]

Username [_____]

Email [_____]

Phone [_____]

Role [Admin ▼]

Password [_____]

Confirm Password [_____]

[Register]

JavaScript: validation
Django: create user + password hashing
REST API: registration request

---

2. Login — `login.html`

FORM ONLY

LOGIN

Username [_____]

Password [_____]

☐ Remember Me

[Login]

Forgot Password?

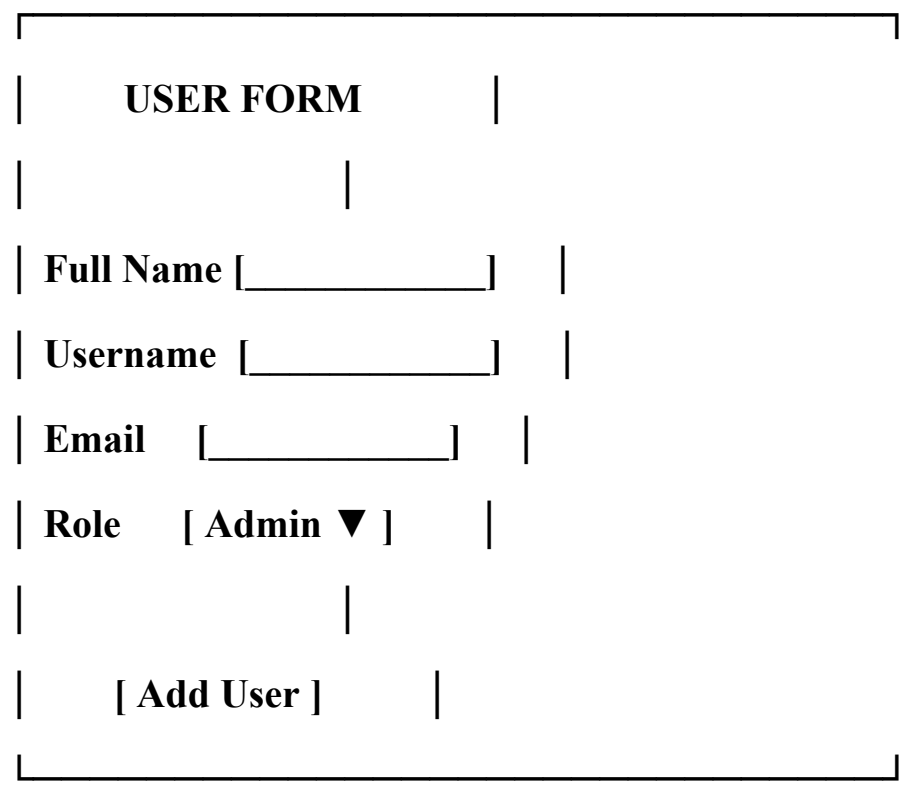
JavaScript: validation
Django: authentication
REST API: login request

---

3. User Management — `users.html`

ONE FORM + ONE TABLE

USER MANAGEMENT



Search: [_____] Role: [All ▼]

ID	Full Name	Username	Role	Actions
1	Anu	anu01	Admin	Edit Delete
2	Rahul	rahul01	Pharmacist	Edit Delete

└──────┴──────────────────┴──────────────────┴──────────────┴──────────────┘

---

4. Medicine Management — `medicines.html`

ONE FORM + ONE TABLE

MEDICINE MANAGEMENT

┌──┐

│ MEDICINE FORM │

│ │

│ Medicine Name [_____] │

│ Generic Name [_____] │

│ Category [_____] │

│ Manufacturer [_____] │

│ Dosage Form [_____] │

│ Strength [_____] │

│ Reorder Level [_____] │

│ Description [_____] │

│ │

[Add Medicine]

Search: [_____] Category: [All ▼]

ID	Medicine	Category	Strength	Actions
1	Paracetamol	Painkiller	500 mg	Edit Delete

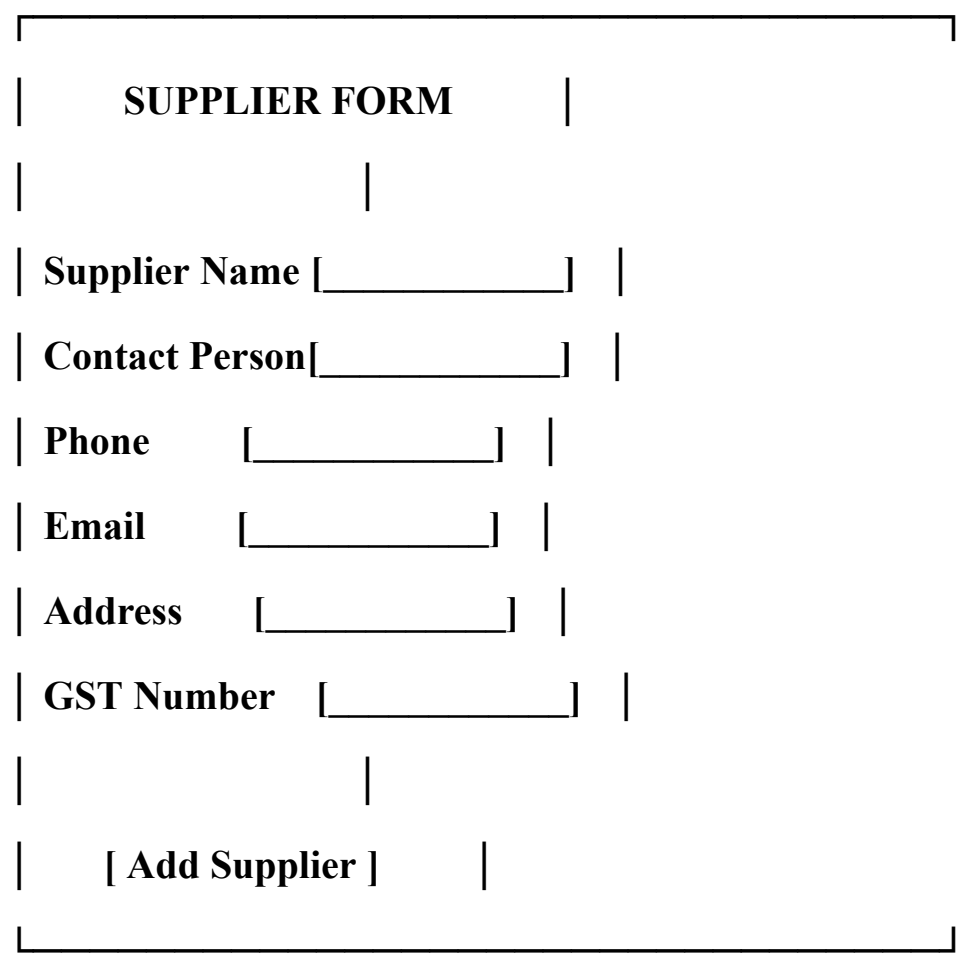
Important: Batch number, expiry and quantity are not medicine-master fields.

---

5. Supplier Management — `suppliers.html`

ONE FORM + ONE TABLE

SUPPLIER MANAGEMENT



Search: [_____]

ID	Supplier	Contact	Phone	Actions
1	ABC Pharma	Anil	98765...	Edit Delete

└──────┴──────────────────┴──────────────────┴──────────────────┴──────────────┘

---

6. Purchase Management — `purchases.html`

ONE FORM + ONE TABLE

PURCHASE MANAGEMENT

┌──┐

│ PURCHASE FORM │

│ │

│ Supplier [ABC Pharma ▼] │

│ Purchase Date [09-09-2026] │

│ Medicine [Paracetamol ▼] │

│ Batch Number [_____] │

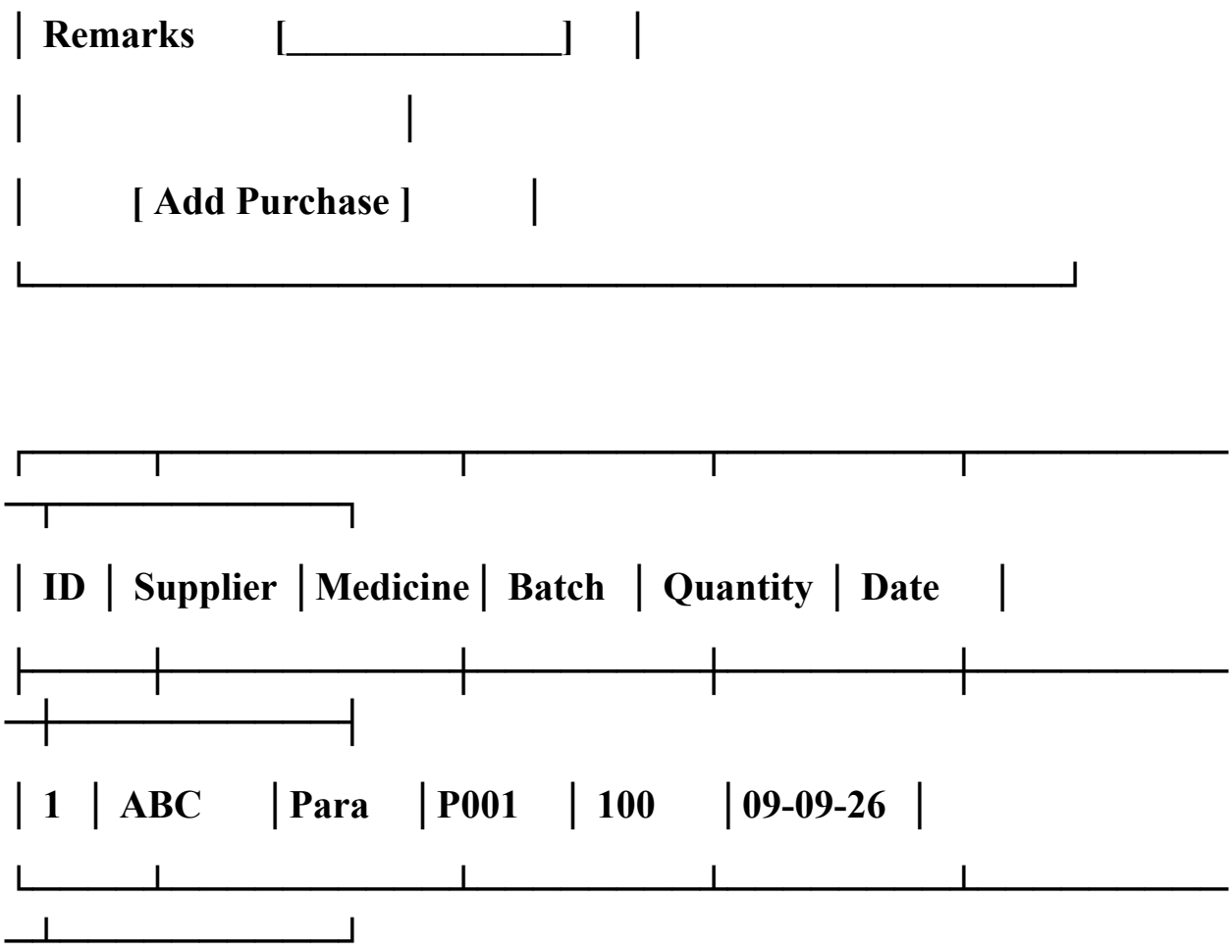
│ Manufacturing [_____] │

│ Expiry Date [_____] │

│ Quantity [_____] │

│ Purchase Price [_____] │

│ Selling Price [_____] │



Backend effect:

Purchase 100

↓

Batch created

↓

Stock +100

↓

Transaction created

---

7. Batch Management — `batches.html`

TABLE ONLY

BATCH MANAGEMENT

Search: [_____]

Medicine: [All ▼]

Expiry: [All ▼]

ID	Medicine	Batch	Mfg Date	Expiry	Quantity
1	Paracetamol	P001	01-01-2026	01-01-2028	100
2	Paracetamol	P002	01-03-2026	01-03-2027	50

A medicine can have multiple batches.

---

8. Inventory / Stock — `inventory.html`

FILTER + TABLE

INVENTORY

Search: [_____]

Category: [All ▼]

Status: [All ▼]

Medicine	Batch	Quantity	Reorder Level	Status
Paracetamol	P001	100	20	IN STOCK
Amoxicillin	A001	10	20	LOW STOCK

No separate form is necessary because stock changes through:

Purchase → Issue → Return

---

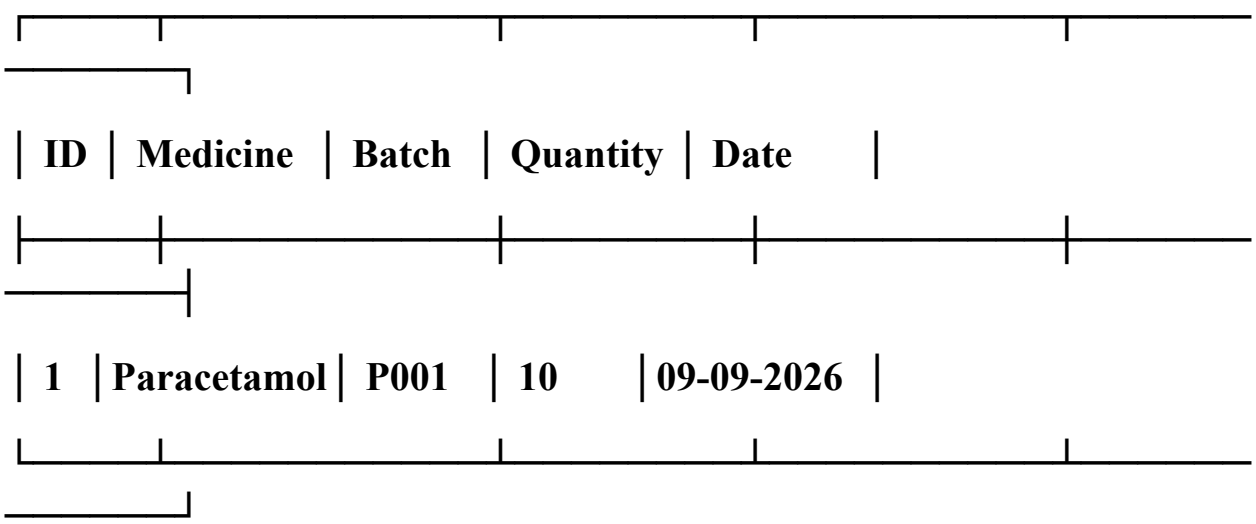
9. Medicine Issue — `issue.html`

ONE FORM + ONE TABLE

MEDICINE ISSUE

```
┌──────────────────────────────────────────────┐
│          ISSUE FORM                          │
│                                              │
│ Medicine     [Paracetamol ▼]                 │
│ Batch        [P001 ▼]                        │
│ Available    [100]                           │
│ Issue Qty    [10]                            │
│ Customer     [_____________]                 │
│ Issue Date   [_____________]                 │
│ Remarks      [_____________]                 │
│                                              │
│          [ Issue Medicine ]                  │
└──────────────────────────────────────────────┘
```

Recent Issues



ID	Medicine	Batch	Quantity	Date
1	Paracetamol	P001	10	09-09-2026

Stock:

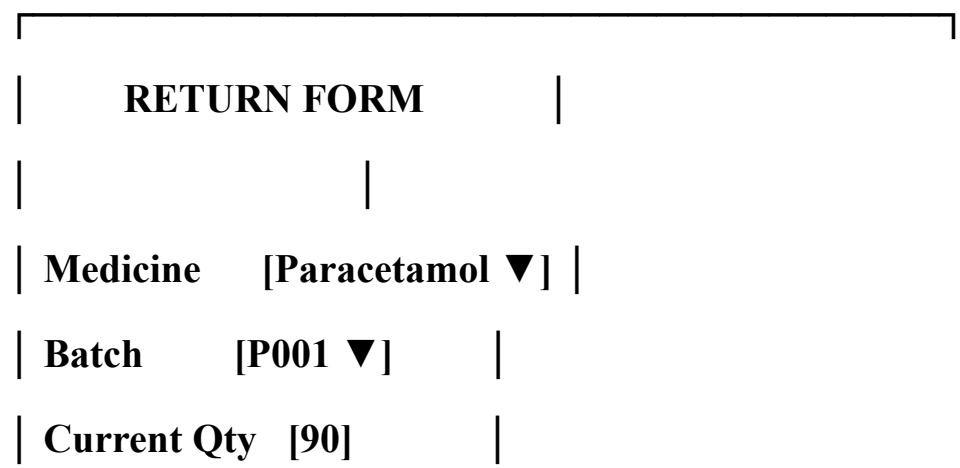
100 - 10 = 90

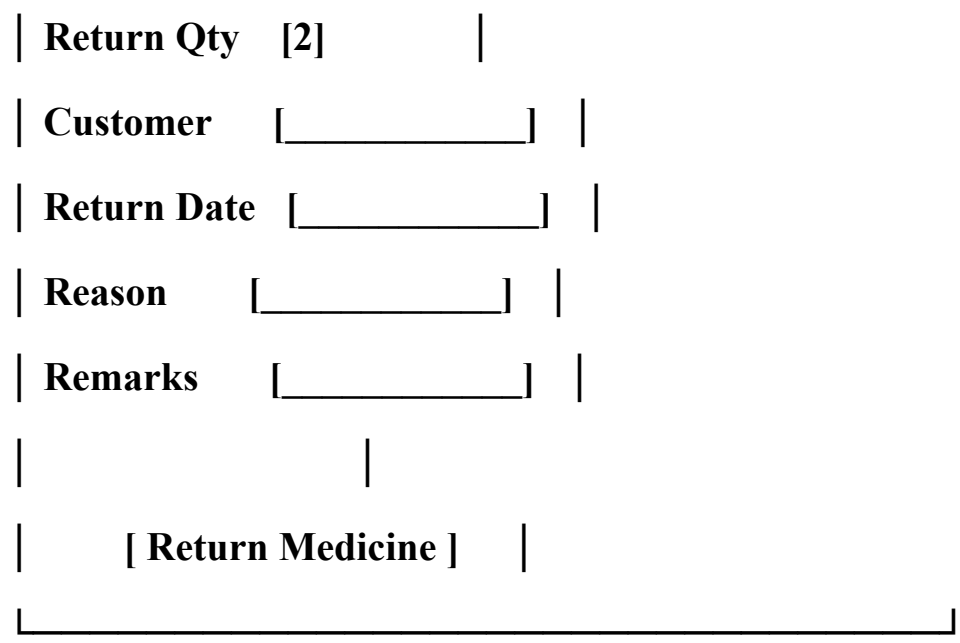
---

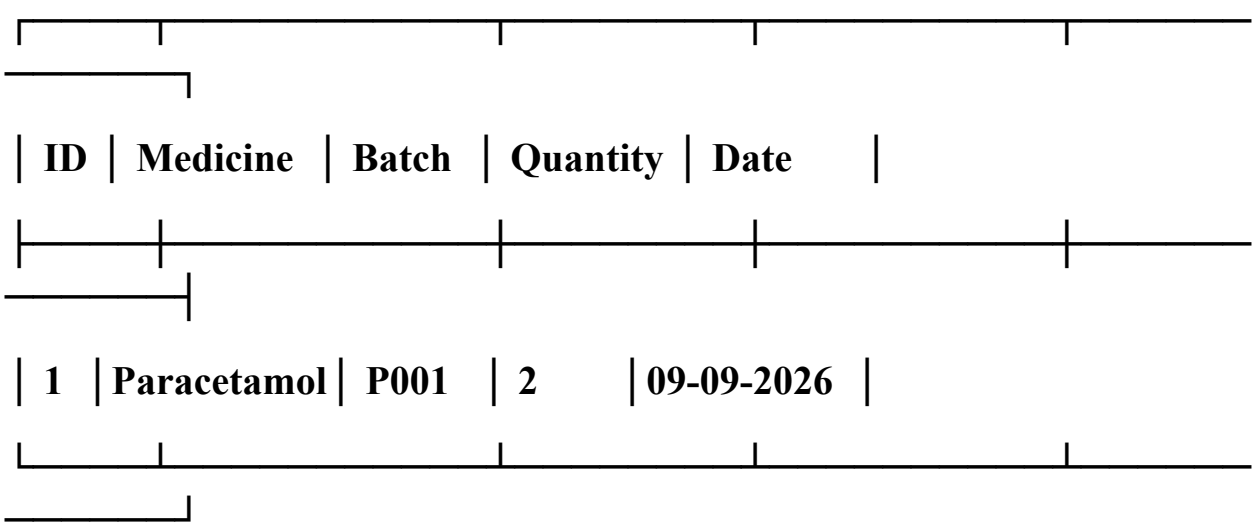
10. Medicine Return — `returns.html`

ONE FORM + ONE TABLE

MEDICINE RETURN







Stock:

90 + 2 = 92

---

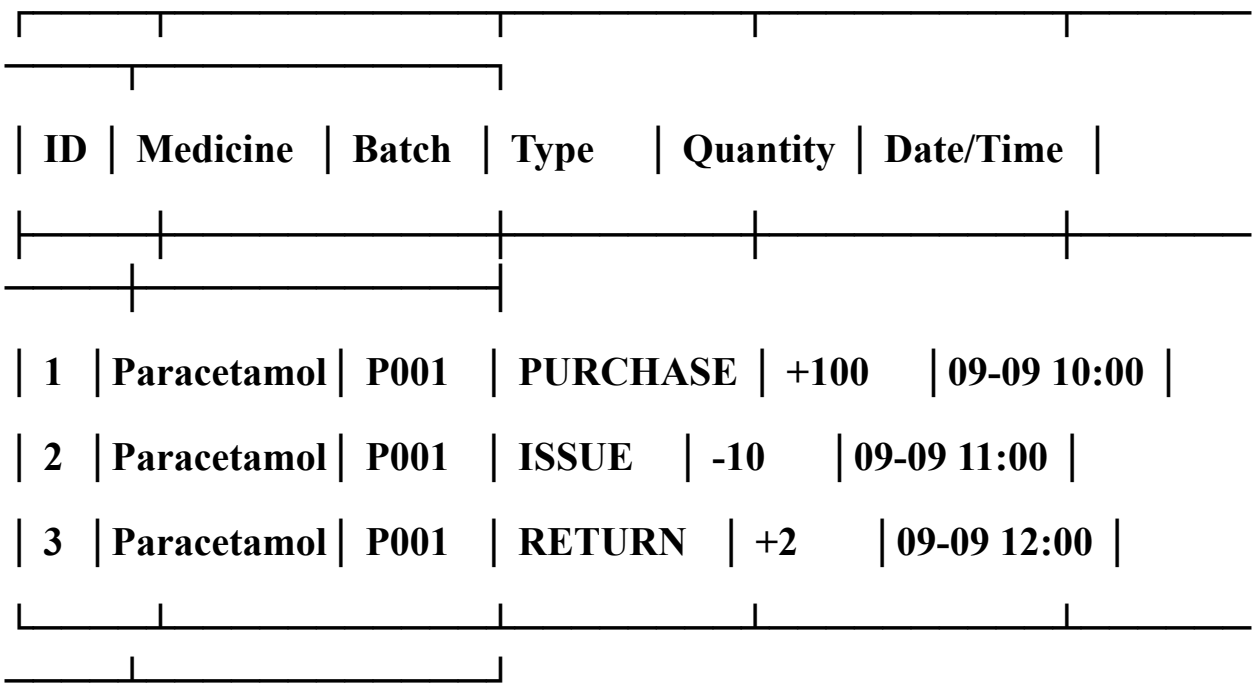
11. Transactions — `transactions.html`

FILTER + TABLE

TRANSACTIONS

Date: [_____] Type: [All ▼]

Medicine: [All ▼]



---

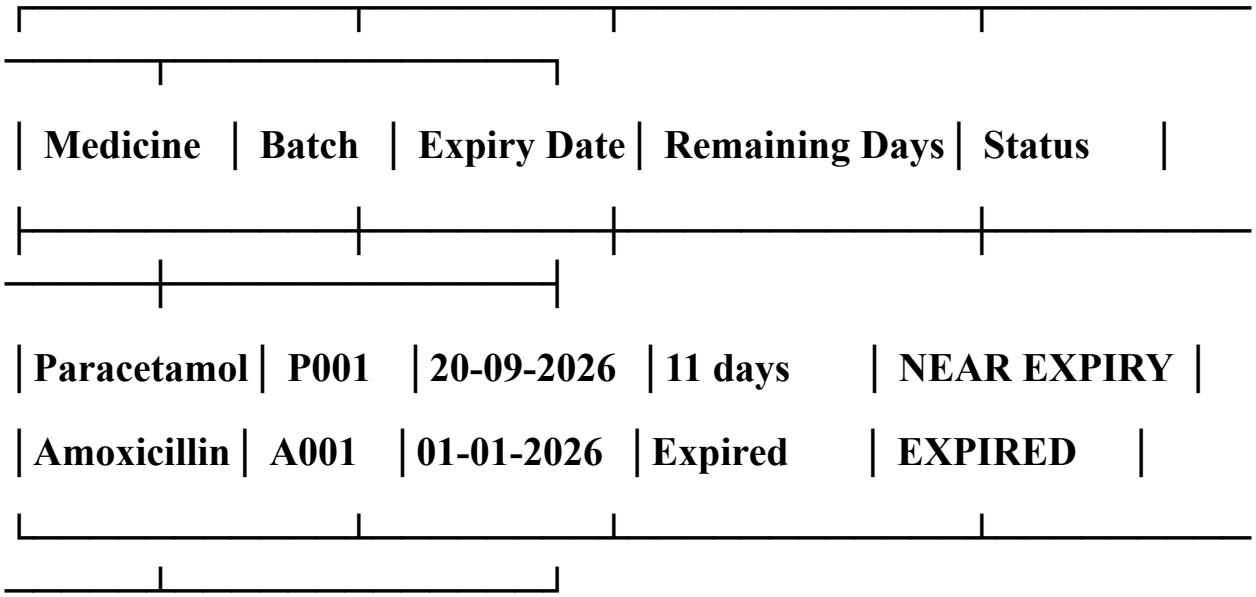
12. Expiry Tracking — `expiry.html`

FILTER + TABLE

EXPIRY TRACKING

Status: [All ▼]

Medicine: [All ▼]



Medicine	Batch	Expiry Date	Remaining Days	Status
Paracetamol	P001	20-09-2026	11 days	NEAR EXPIRY
Amoxicillin	A001	01-01-2026	Expired	EXPIRED

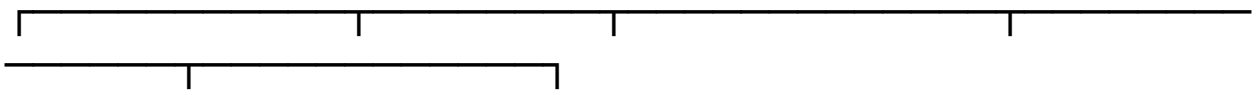
JavaScript can calculate remaining days initially; Django/API should provide authoritative values once backend is connected.

---

13. Low Stock — `low-stock.html`

TABLE ONLY

LOW STOCK



Medicine	Batch	Current Qty	Reorder Level	Status
Amoxicillin	A001	10	20	LOW STOCK
Cetirizine	C001	5	15	LOW STOCK

Logic:

Current Quantity <= Reorder Level

↓

LOW STOCK

---

14. Dashboard — `dashboard.html`

CARDS + TABLE + ALERTS

DASHBOARD

Medicines	Batches	Total Stock
125	240	5,420

Low Stock	Near Expiry	Expired
8	12	3

⚠ LOW STOCK ALERT

Amoxicillin A001 — 10 units remaining

RECENT TRANSACTIONS

Medicine	Batch	Type	Quantity
Paracetamol	P001	ISSUE	-10
Amoxicillin	A001	PURCHASE	+100

---

15. Reports — `reports.html`

FILTER FORM + TABLE

REPORTS

REPORT FILTER

Report Type [Stock ▼]

From Date [_____]

To Date [_____]

Medicine [All ▼]

[Generate Report]

REPORT RESULT

Medicine	Batch	Quantity	Status
Paracetamol	P001	92	IN STOCK
Amoxicillin	A001	10	LOW

[Export PDF] [Export Excel]

Final HTML File List

frontend/

│

├── login.html

├── register.html

├── dashboard.html

├── users.html

├── medicines.html

├── suppliers.html

├── purchases.html

├── batches.html

├── inventory.html

├── issue.html

├── returns.html

├── transactions.html

├── expiry.html

├── low-stock.html

└── reports.html

The important rule for your students

HTML

↓

Creates FORM + TABLE structure

CSS

↓

Styles the SAME FORM + TABLE

Bootstrap

↓

Makes the SAME FORM + TABLE responsive

JavaScript

↓

Makes the SAME FORM + TABLE interactive

React

↓

Converts the SAME UI into components/pages

Django

↓

Stores the SAME form data in SQLite

REST API

↓

Connects React ↔ Django

WebSocket

↓

Updates the SAME tables/dashboard in real time

Complete Technology Mapping

Module	Database / Django Fields	HTML	Bootstrap	JavaScript
1. Registration	name, username, email, phone, role, password	Form	Form-control, button, card	Validation, password match
2. Login	username, password	Form	Card, input-group, button	Validation
3. Users	full_name, username, email, role	Form + Table	Table, badge, modal	Search, filter, edit, delete, validation
4. Medicines	name, generic_name, category, manufacturer, dosage_form, strength, reorder_level, description	Form + Table	Form, table, badge, modal	Search, filter, edit, delete, validation
5. Suppliers	supplier_name, contact_person, phone, email, address, gst_number	Form + Table	Form, table, modal	Search, validation, edit/delete
6. Purchases	supplier, medicine, batch, mfg_date, expiry_date, quantity,	Form + Table	Form, select, table	Validation, price calculation,

	purchase_price, selling_price, date			**expiry validation**
7. Batches	**medicine, batch_number, mfg_date, expiry_date, quantity, purchase_price, selling_price**	**Table**	**Table, badge, pagination**	**Search, filter, expiry calculation**
8. Inventory	**medicine/batch, quantity, reorder_level**	**Filter + Table**	**Cards, table, badges**	**Search, filter, stock status**
9. Medicine Issue	**medicine, batch, issue_quantity, customer, date, remarks**	**Form + Table**	**Form, table, alert**	**Check available stock, calculate remaining**
10. Medicine Return	**medicine, batch, return_quantity, customer, reason, date**	**Form + Table**	**Form, table, alert**	**Validate return quantity, stock calculation**
11. Transactions	**medicine, batch, type, quantity, user, date, remarks**	**Filter + Table**	**Table, badges, dropdown**	**Search/filter**
12. Expiry Tracking	**batch, expiry_date**	**Filter + Table**	**Table, badges, alerts**	**Remaining-days calculation**

13. Low Stock	**quantity, reorder_level**	**Table**	**Alert, badge, table**	**Stock status**
14. Dashboard	**Aggregated database data**	**Cards + Tables + Alerts**	**Cards, progress, alerts**	**Dynamic dashboard updates**
15. Reports	**Data from all tables**	**Filter + Table**	**Form, table, buttons**	**Filter, export trigger**

---

1. USERS MODULE

Database / Django Model

```
class UserProfile(models.Model):
    full_name = models.CharField(max_length=100)
    username = models.CharField(max_length=50, unique=True)
    email = models.EmailField(unique=True)
    role = models.CharField(max_length=30)
```

HTML

One file: `users.html`

FORM
├── Full Name
├── Username
├── Email
└── Role

TABLE

├── ID
├── Full Name
├── Username
├── Email
├── Role
└── Edit / Delete

Bootstrap

- `form-control`
- `form-select`
- `btn`
- `table`
- `table-striped`
- `badge`
- `modal`

JavaScript

Add User
↓
Validate fields
↓
Search users
↓
Filter by role
↓
Edit user
↓
Delete user

---

2. MEDICINE MODULE

Database / Django

```
class Medicine(models.Model):
    name = models.CharField(max_length=100)
    generic_name = models.CharField(max_length=100)
    category = models.CharField(max_length=100)
    manufacturer = models.CharField(max_length=100)
    dosage_form = models.CharField(max_length=50)
    strength = models.CharField(max_length=50)
    reorder_level = models.IntegerField(default=0)
    description = models.TextField(blank=True)
```

HTML

FORM

- Medicine Name
- Generic Name
- Category
- Manufacturer
- Dosage Form
- Strength
- Reorder Level
- Description

TABLE

- ID
- Medicine
- Category
- Manufacturer
- Strength
- Actions

Bootstrap

form-control
form-select
table
table-hover
badge
btn-primary
btn-warning
btn-danger

JavaScript

Validation
Search medicine
Filter category
Edit
Delete

---

3. SUPPLIER MODULE

Database

```
class Supplier(models.Model):
    supplier_name = models.CharField(max_length=100)
    contact_person = models.CharField(max_length=100)
    phone = models.CharField(max_length=15)
    email = models.EmailField()
    address = models.TextField()
    gst_number = models.CharField(max_length=30, blank=True)
```

Output

```
FORM
 ├── Supplier Name
 ├── Contact Person
```

├── Phone
├── Email
├── Address
└── GST Number

TABLE
├── ID
├── Supplier
├── Contact
├── Phone
├── Email
└── Actions

---

4. PURCHASE MODULE

This is an important module because purchase changes stock.

Database

```
class Purchase(models.Model):
    supplier = models.ForeignKey(Supplier,
on_delete=models.CASCADE)
    medicine = models.ForeignKey(Medicine,
on_delete=models.CASCADE)
    batch_number = models.CharField(max_length=50)
    manufacturing_date = models.DateField()
    expiry_date = models.DateField()
    quantity = models.IntegerField()
    purchase_price = models.DecimalField(max_digits=10,
decimal_places=2)
    selling_price = models.DecimalField(max_digits=10,
decimal_places=2)
```

```
    purchase_date = models.DateField()
    remarks = models.TextField(blank=True)
```

HTML

FORM

- Supplier
- Medicine
- Batch Number
- Manufacturing Date
- Expiry Date
- Quantity
- Purchase Price
- Selling Price
- Remarks

TABLE

- Supplier
- Medicine
- Batch
- Quantity
- Price
- Date

JavaScript

Select medicine

↓

Enter quantity

↓

Enter price

↓

Validate expiry

↓

Submit purchase

Django

Purchase
↓
Create / Update Batch
↓
Increase Stock
↓
Create Transaction

---

5. BATCH MODULE

Database

```
class Batch(models.Model):
    medicine = models.ForeignKey(Medicine,
on_delete=models.CASCADE)
    batch_number = models.CharField(max_length=50)
    manufacturing_date = models.DateField()
    expiry_date = models.DateField()
    quantity = models.IntegerField(default=0)
    purchase_price = models.DecimalField(max_digits=10,
decimal_places=2)
    selling_price = models.DecimalField(max_digits=10,
decimal_places=2)
```

Output

TABLE

Medicine | Batch | Mfg Date | Expiry | Quantity | Status

Example:

Paracetamol | P001 | 01-01-26 | 20-09-26 | 100 | Normal

Paracetamol | P002 | 01-03-26 | 15-09-26 | 50 | Near Expiry

JavaScript

Calculate:

Expiry Date - Today
↓
Remaining Days
↓
NORMAL / NEAR EXPIRY / EXPIRED

---

6. INVENTORY MODULE

Database source

Inventory can use:

Batch.quantity

instead of creating another stock table for a simpler student project.

Output

Medicine | Batch | Available Qty | Reorder Level | Status

Example:

Paracetamol | P001 | 100 | 20 | IN STOCK
Amoxicillin | A001 | 10 | 20 | LOW STOCK

Bootstrap

Use:

card
table

badge
alert

---

7. ISSUE MODULE

Database

```
class Issue(models.Model):
    medicine = models.ForeignKey(Medicine,
on_delete=models.CASCADE)
    batch = models.ForeignKey(Batch, on_delete=models.CASCADE)
    quantity = models.IntegerField()
    customer_name = models.CharField(max_length=100, blank=True)
    issue_date = models.DateTimeField(auto_now_add=True)
    remarks = models.TextField(blank=True)
```

JavaScript

Available = 100

Issue = 10

Remaining = 100 - 10

Remaining = 90

If:

Issue Quantity > Available Quantity

show:

❌ Insufficient Stock

---

8. RETURN MODULE

JavaScript

Current Stock = 90
Return = 2

90 + 2 = 92

Django

Return
↓
Increase Batch.quantity
↓
Create RETURN transaction

---

9. TRANSACTION MODULE

Database

```
class Transaction(models.Model):
    medicine = models.ForeignKey(Medicine,
on_delete=models.CASCADE)
    batch = models.ForeignKey(Batch, on_delete=models.CASCADE)
    transaction_type = models.CharField(max_length=20)
    quantity = models.IntegerField()
    user = models.ForeignKey(User, on_delete=models.CASCADE)
    created_at = models.DateTimeField(auto_now_add=True)
    remarks = models.TextField(blank=True)
```

Types:

PURCHASE

ISSUE
RETURN
ADJUSTMENT

Output

ID | Medicine | Batch | Type | Quantity | User | Date

---

10. EXPIRY MODULE

Database

Expiry comes from:

Batch.expiry_date

JavaScript

Today = 09-09-2026
Expiry = 20-09-2026

Remaining = 11 days

Then:

0–30 days → NEAR EXPIRY
Past date → EXPIRED
More than 30 → NORMAL

---

11. LOW STOCK MODULE

Logic

Batch.quantity <= Medicine.reorder_level

Example:

Current = 10
Reorder Level = 20

10 <= 20
↓
LOW STOCK

Output

Medicine | Batch | Current Stock | Reorder Level | Status

Bootstrap:

badge-danger
alert-danger
table

---

12. DASHBOARD

Database data comes from all modules

Medicine
Batch
Inventory
Transaction
Expiry
Low Stock

Output



└──────────────────────┘

┌──────────────────────┐
│ Total Stock │
│ 5,420 │
└──────────────────────┘

┌──────────────────────┐
│ Low Stock │
│ 8 │
└──────────────────────┘

┌──────────────────────┐
│ Near Expiry │
│ 12 │
└──────────────────────┘

Then:

Recent Transactions → TABLE

Low Stock → ALERT

Near Expiry → ALERT

---

13. REPORT MODULE

Form

Report Type [Stock ▼]
From Date [_____]
To Date [_____]
Medicine [All ▼]

[Generate Report]

Output

Medicine | Batch | Quantity | Status

Bootstrap:

form
select
button
table
pagination

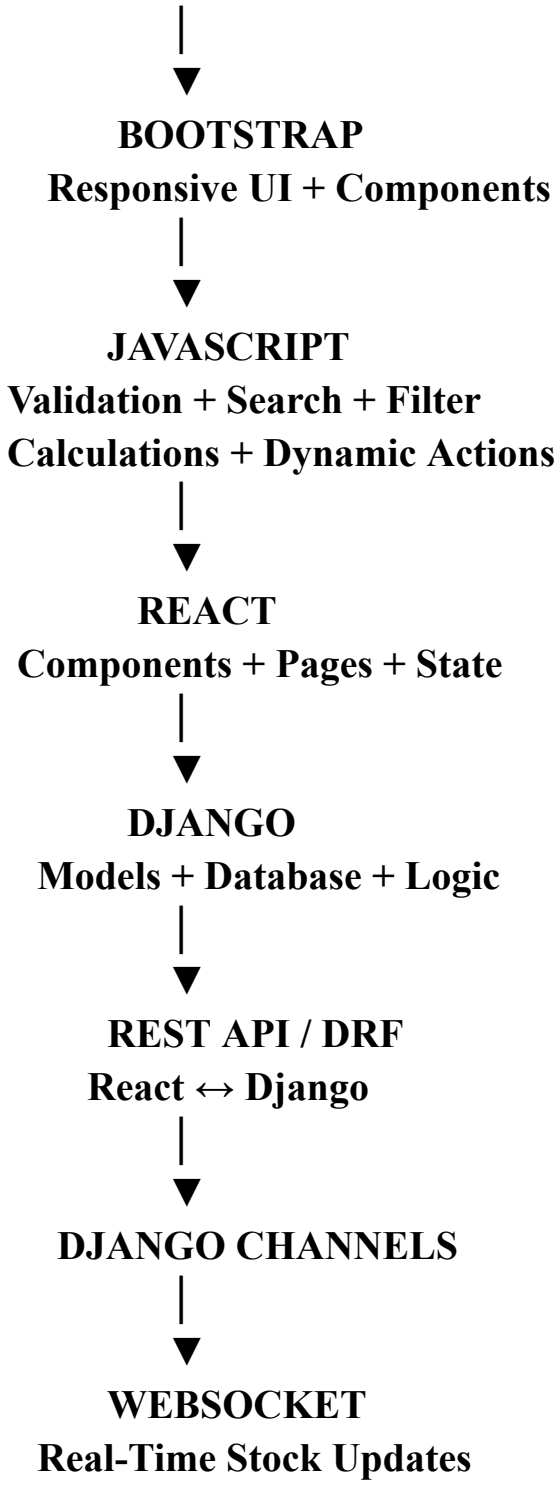
JavaScript:

Date filter
Medicine filter
Report type filter

Overall Student Workflow

This is the best way to divide the same project among students:

MEDICINE INVENTORY SYSTEM
|
▼
HTML
Forms + Tables + Pages
|
▼
CSS
Design & Styling



Most important distinction

Database/Django fields = what information is stored.

HTML = how the form/table is structured.

Bootstrap = how the interface looks and responds.

JavaScript = what happens in the browser.

React = converts the frontend into reusable components.

Django = handles database + backend logic.

REST API = connects React and Django.

WebSocket = gives real-time updates.